

RAG em Python puro

O que é e como funciona: por dentro do repositório `webinarioOllamaRAG` — do Ollama local à geração remota com fallback, sem perder o controle de cada peça.

213

chunks no Corpus
Oficial

4

artigos científicos em
português

3

providers de geração
em fallback

119

testes automatizados

Ensaio no CIIA: 21/09/2026 · Turma aberta: 28/09/2026

Stack: Python · Ollama (bge-m3) · Chroma · Streamlit · NVIDIA / Gemini / OpenAI

RAG em um minuto: indexar, recuperar, gerar

Retrieval-Augmented Generation: em vez de confiar no que o modelo memorizou, o sistema busca trechos relevantes nos seus documentos e entrega só esses trechos ao LLM, que responde com base neles.

1

Indexar

Feito uma vez, offline

- PDF → texto por página (pypdf)
- Texto → chunks de até 1000 caracteres
- Chunk → vetor de 1024 números (bge-m3)
- Vetores + metadados → coleção Chroma

`scripts/02_indexar_hibrido.py`

2

Recuperar

A cada pergunta

- Pergunta → vetor com o mesmo modelo
- Chroma devolve os k vizinhos mais próximos
- Filtros por metadados (ano, tema)
- Limiar de distância descarta o que é fraco

`openai_rag.py · OpenAIRAG.buscar()`

3

Gerar

A cada pergunta, em streaming

- LLM recebe só os trechos numerados [n]
- Responde citando [1], [2]...
- Sem evidência: recusa com frase fixa
- Fallback entre NVIDIA, Gemini e OpenAI

`generation_router.py · GenerationRouter`

Por que não perguntar direto ao LLM? conhecimento atualizado sem re-treinar o modelo · resposta rastreável até o arquivo e a página · recusa explícita quando não há evidência, em vez de alucinação.

Ponto de partida: máquina modesta, modelo que não citava

7,9 GB

de RAM na máquina da demo

0

GPU: só CPU i5-8250U

1,5 B

parâmetros no qwen2.5 local

510 s

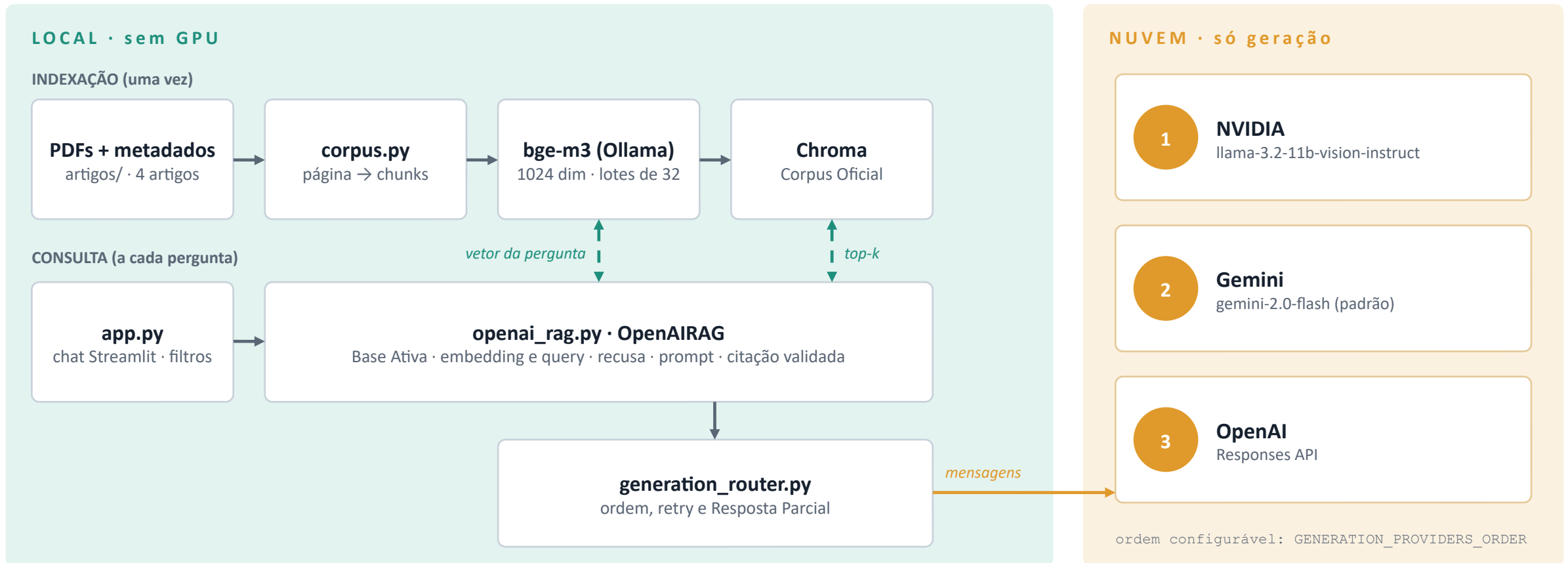
para indexar 213 chunks em CPU

Etapa	Onde roda	Por quê
Embeddings (bge-m3)	Local, via Ollama	Documentos e vetores não saem da máquina; sem cota nem custo por chamada
Índice vetorial (Chroma)	Local, em disco	Persistente, versionado e com rollback por manifesto
Geração do texto	Nuvem: NVIDIA → Gemini → OpenAI	Qualidade de resposta e obediência ao formato de citação [n]
Fallback para geração local	Não existe	Só por rollback explícito para a tag legacy-pre-openai

Achado A1-08 da auditoria: a geração local era lenta e o modelo pequeno não emitia marcadores [n]. Sem marcador, não existe Fonte Citada.

Decisão: manter o retrieval local e didático e levar só a geração para a nuvem. Atenção: a pergunta e até 3 trechos ainda são enviados ao provider de geração.

Arquitetura híbrida: tudo local, menos a redação da resposta



Contrato mínimo de geração: `transmitir(mensagens)` → iterador de deltas de texto. Trocar ou reordenar providers não muda nada acima do roteador. Índice de Sessão (uploads) usa o mesmo bge-m3 num Chroma efêmero; cada pergunta consulta uma única Base Ativa.

O corpus é meta: um RAG que responde sobre artigos de RAG

RAG jurídico

Brakes et al. · ERI-GO 2025 · tema aplicações

38

chunks

Segmentação ancorada: corta o texto legal em unidades com sentido (artigo, parágrafo, inciso) e enriquece cada trecho com pré-contexto. Filtros estruturados: restringem a busca por metadados antes de comparar vetores.

RAG em grafos

Xavier & Soares · Minicursos SBBD 2024 · fundamentos

71

chunks

Grafo de conhecimento: o saber vira entidades e relações (ex.: Neo4j). A recuperação segue caminhos entre entidades, útil para perguntas que ligam vários fatos, e não só vizinhança de vetores.

Embeddings para português

Medeiros & Oliveira · SEMISH 2025 · avaliação

50

chunks

Embedding é o modelo que transforma texto em vetor. O artigo compara modelos de embedding e LLMs em RAG para português, base para discutir a escolha do bge-m3, multilíngue.

Ajuste fino (SFT)

Olivera et al. · SBBD 2025 · retrieval

54

chunks

Ajuste fino sequencial (SFT) ensina modelos pequenos a usar melhor o contexto recuperado: uma alternativa a depender de modelos grandes na nuvem.

A ideia do artigo jurídico está dentro da aplicação

- Os filtros de ano e tema da interface viram um where no Chroma
- O filtro restringe o espaço e só então a similaridade decide
- Mesmo princípio dos filtros estruturados de Brakes et al.
- Todos os 213 chunks estão em português (idioma pt em 213/213)
- Antes eram 8 artigos e 661 chunks, 6 deles em inglês; a coleção antiga continua no Chroma para rollback

```
where = {"ano": {"$gte": 2024},  
        "tema": {"$in": ["aplicacoes"]}}
```

Do PDF ao índice: os termos que aparecem no código

Chunk

Pedaço de texto que vira a unidade de busca. Aqui: até 1000 caracteres, cortado no último espaço para não partir palavras.

Sobreposição (overlap)

150 caracteres repetidos entre chunks vizinhos, para que uma ideia cortada na borda apareça inteira em pelo menos um deles.

Chunk por página

Cada chunk nasce de uma única página do PDF. Isso permite citar o arquivo e a página exata da evidência.

Chunk de resumo

Um chunk extra por artigo com título + resumo, tipo_chunk = resumo. Na interface aparece como "p. 0".

Metadados

arquivo, título, autores, ano, veículo, tema, idioma, resumo, página, chunk_id e tipo_chunk. Servem para filtrar e para citar.

Embedding

Vetor de 1024 números gerado pelo bge-m3. Textos com sentido parecido ficam próximos; distância por cosseno no Chroma.

Anatomia de um chunk_id

brakes2025_rag_juridico

arquivo (stem)

-p003

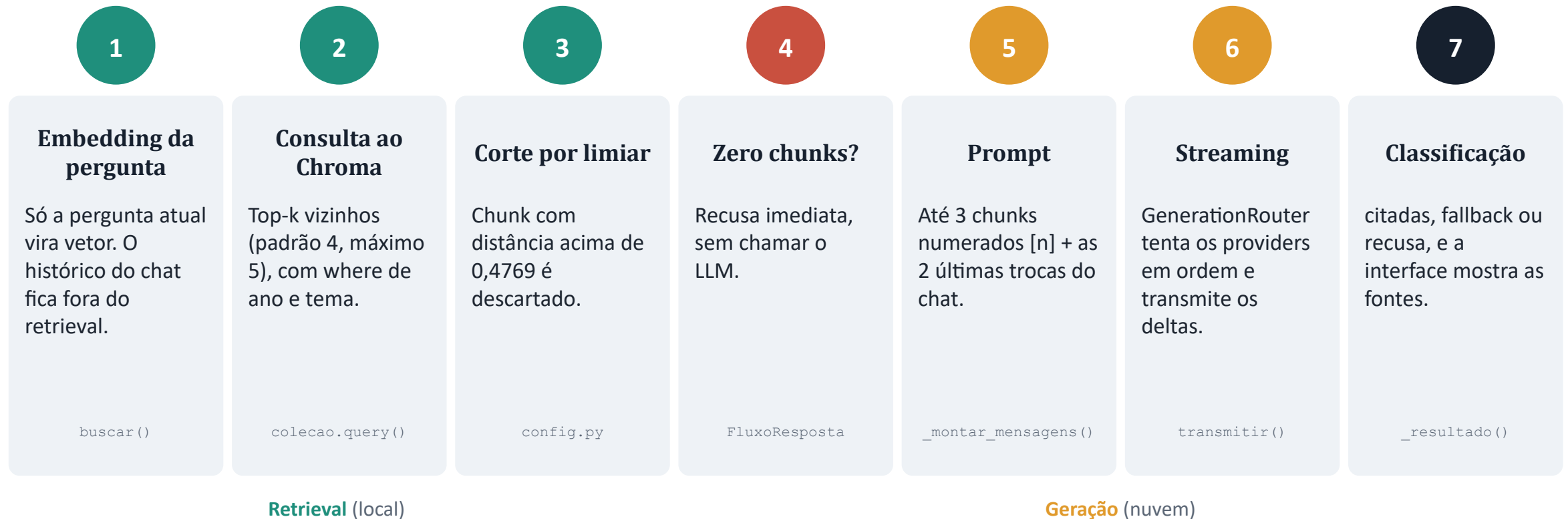
página 3

-c01

1º chunk da
página

Resultado: 213 chunks (54 + 50 + 38 + 71), 213 embeddings, 0 vazios, 0 duplicados, 0 caracteres corrompidos. Resumos usam o id <stem>-resumo.

O caminho de uma pergunta, passo a passo



Por que o histórico fica fora da busca? Para que a mesma pergunta recupere sempre os mesmos trechos. O histórico só ajuda o LLM a entender referências como "e o segundo artigo?". Embedding e busca rodam uma única vez por pergunta, mesmo se o provider trocar.

O prompt que obriga o modelo a ficar nos trechos

```
# openai_rag.py · _montar_mensagens()
```

system:

Responda em português usando SOMENTE os trechos fornecidos. Se a resposta não estiver nos trechos, responda exatamente: Não encontrei essa informação nos documentos. Cite cada afirmação sustentada usando o número do trecho entre colchetes, como [1].

```
+ histórico[-4:] # 2 últimas trocas
```

user:

Trechos:

```
[1] (brakes2025_rag_juridico.pdf, p. 3, 2025)
```

```
<texto do chunk>
```

```
[2] (...) [3] (...) # no máximo 3
```

Pergunta: <pergunta atual>

1

SOMENTE os trechos

Proíbe conhecimento externo. É o que chamamos de grounding estrito.

2

Frase de recusa exata

Uma string fixa pode ser detectada no pós-processamento e contada nas métricas.

3

[n] como contrato

O número liga cada afirmação a um chunk. É a base da citação validada.

4

No máximo 3 trechos

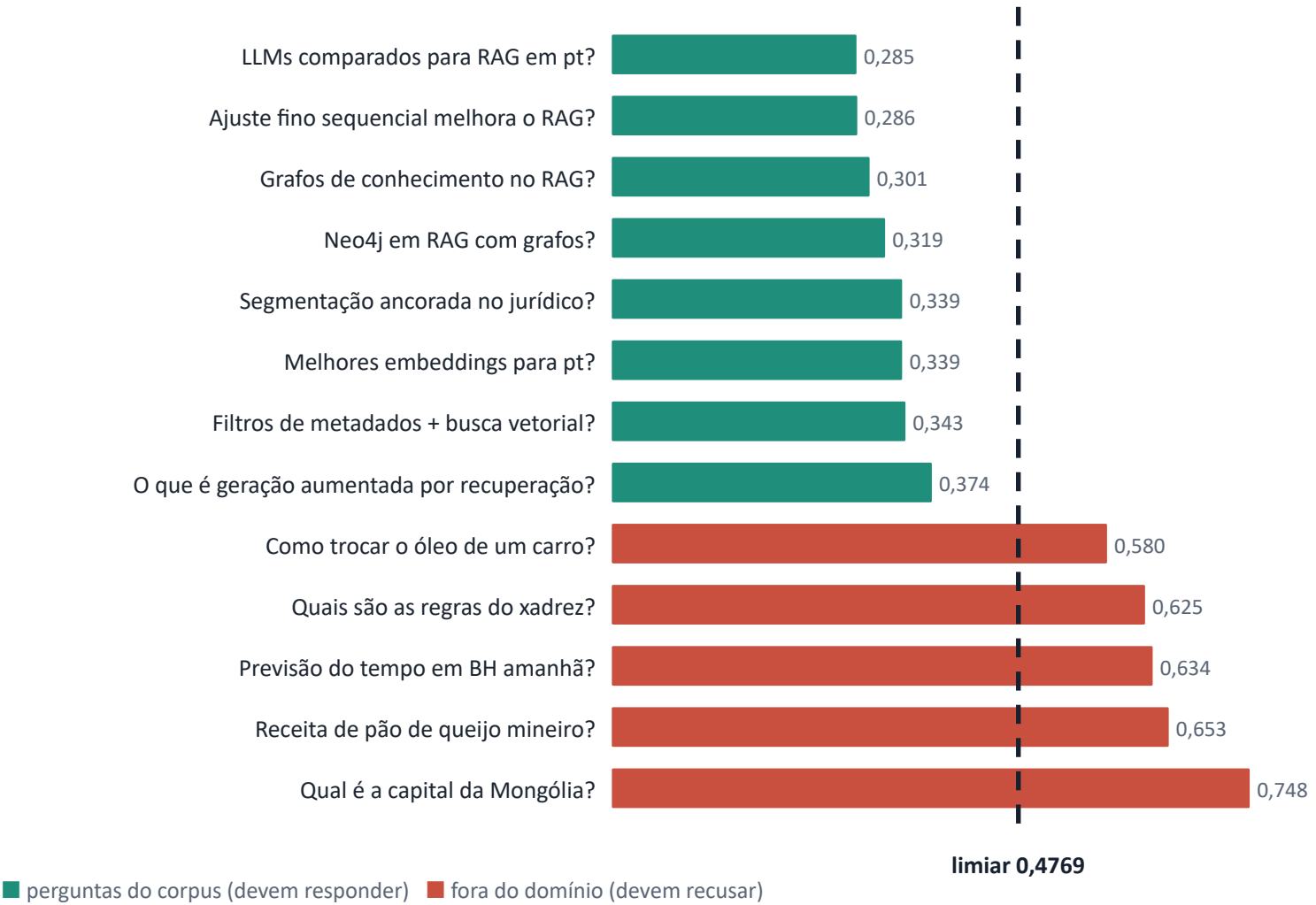
Menos ruído e prompt menor; a citação só aceita [1] a [3].

5

Histórico só na geração

As 2 últimas trocas resolvem referências, sem contaminar a busca.

Recusar antes do LLM: um limiar calibrado



0,4769

ponto médio entre a maior distância positiva (0,3738) e a menor negativa (0,5800): margem de 0,103 para cada lado

- Custo zero:** sem token e sem chamada de rede
- Latência mínima:** só o embedding da pergunta
- Sem alucinação:** o LLM nem é consultado
- Calibração exige** que o top-1 venha do artigo esperado, não só distância baixa
- Recalibrar** a cada mudança de corpus: antes era 0,5100

Citação validada: Chunk Recuperado não é Fonte Citada

```
_CITACAO_RE = re.compile(r"\\(\\d+)\\")

def _fontes_citadas(texto, chunks):
    indices = []
    for m in _CITACAO_RE.finditer(texto):
        i = int(m.group(1))
        if 1 <= i <= min(len(chunks), 3) \
            and i not in indices:
            indices.append(i)
    return [chunks[i - 1] for i in indices]
```

Um [7] quando só existem 3 trechos é ignorado. Citação repetida conta uma vez.

Princípio: o LLM é um componente não confiável. Ter recuperado um trecho não prova que a resposta veio dele.

Exemplo: "...usa pré-contexto [1] e filtros [2][7]." → Fontes Citadas: [1] e [2]; o [7] é descartado.

Classe	Quando acontece	O que a interface mostra
citadas	Há pelo menos um [n] válido no texto	"Fontes Citadas": arquivo, página e trecho
fallback	Resposta sem nenhum marcador válido	"Chunks Recuperados": evidência, não citação
recusa	Frase de recusa ou zero chunks	Nenhuma fonte

5 de 5

citações corretas de brakes2025_rag_juridico.pdf pela NVIDIA na pergunta de citação (matriz, cadeia e interface)

4 de 4

respostas sem [n] no filtro por tema: a interface mostrou fallback em vez de inventar citação

Três providers, uma resposta: fallback antes do primeiro token

Antes do primeiro token

- 1 Chama o provider da vez (NVIDIA).
- 2 Erro temporário (408, 429, 5xx, 529) com retry-after de até 2 s? Espera e repete uma única vez.
- 3 Ainda falhou, ou o erro não é temporário? Passa para o próximo (Gemini, depois OpenAI).
- 4 Todos falharam: mensagem de erro acionável, sem expor o corpo da resposta externa.

Depois do primeiro token

- Nunca troca de provider: seriam duas vezes na mesma resposta, e o usuário já leu parte do texto
- O texto recebido é preservado e ganha o aviso: "Resposta Parcial: a geração foi interrompida antes da conclusão."
- A resposta sai marcada com status Resposta Parcial, e isso vai para o log

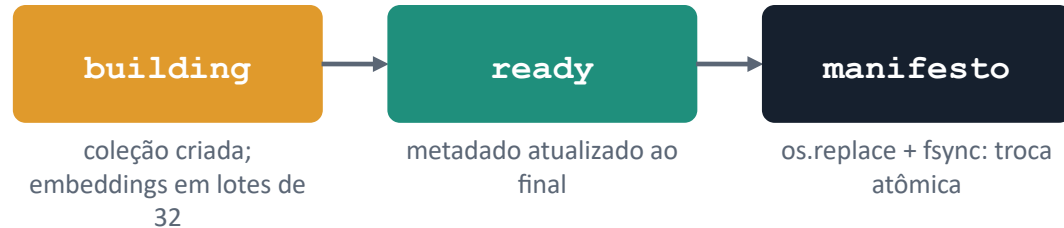
E por que não refazer tudo com outro provider? Custo e latência dobrados, e uma resposta diferente da que o usuário já começou a ler.

```
# .streamlit/secrets.toml ou ambiente
GENERATION_PROVIDERS_ORDER = "nvidia,gemini,openai"
# provider omitido = desativado; nome repetido = erro
```

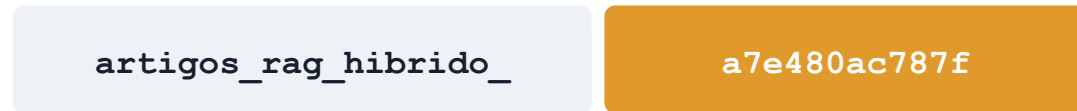
O que vai para o log (rag.geracao): provider, modelo, tentativa, tempo até o primeiro token, fallback usado, recusa e resposta parcial. **Nunca:** chave, prompt, texto de chunk ou texto da resposta.

Um índice que se recusa a ser usado errado

Publicação da coleção



Nome da coleção = identidade do conteúdo



12 primeiros caracteres do SHA-256 da identidade do índice. Mudou o corpus ou o modelo, muda o nome: coleções nunca são sobrescritas.

Rollback = trocar o manifesto. A coleção anterior (a280e65e16ee: 8 artigos, 661 chunks) continua no Chroma. Nenhuma coleção é apagada.

Antes de cada consulta, OpenAIRAG.buscar() confere

✓	provedor_embedding	Ollama
✓	modelo_embedding	bge-m3
✓	dimensao_embedding	1024
✓	versao_colecao	bge-m3-v1
✓	status	ready

Também confere a dimensão do vetor da pergunta. Qualquer divergência interrompe a consulta com:

"A Base Ativa é incompatível (modelo_embedding). Execute a reindexação explícita antes de consultar."

Frameworks prontos × feito à mão

Alternativa	O que entrega pronto	O que esconde ou exige	O que este projeto faz no lugar
LangChain / LlamaIndex	Loaders, splitters, retrievers e cadeias; troca de LLM em uma linha	Prompt, chunking e classificação da resposta ficam dentro de abstrações	Chunking por página, prompt explícito e classes citadas / fallback / recusa em ~250 linhas
Haystack	Pipelines declarativos, componentes e avaliação	Mais uma camada de configuração para aprender	Checagem de compatibilidade da coleção e limiar calibrado com perguntas reais
NVIDIA RAG Blueprint	Stack completa: ingestão, reranking, guardrails, observabilidade	GPU e infraestrutura de containers	Roda em notebook sem GPU; usa a NVIDIA só como endpoint de geração
Assistants / File Search (OpenAI, Gemini)	Indexação, busca e citação gerenciadas	Caixa-preta: sem controle de chunk, limiar ou recusa; dados no fornecedor	Corpus e vetores locais; troca de fornecedor por variável de ambiente

Transparência

cada passo é uma função legível

Recusa determinística

antes do LLM, por regra

Citação validada

marcador [n] checado no código

Trilha auditável

ADR → issue → teste → evidência

O preço de construir à mão, e quando não vale a pena

O que custou

- Reimplementar roteador, retry e streaming para três SDKs diferentes
- Paridade de erros entre SDKs virou um arquivo de teste próprio
- Documentação desalinhada exige revisão contínua (o CLAUDE.md ficou atrás do gate)
- Default de configuração não provado (Gemini 2.0-flash no código)

O que ainda não tem

- Reranking depois da busca vetorial
- Guardrails de entrada e saída
- Avaliação contínua: SHAP e RAGAS ficaram para depois do ensaio (#52)
- OCR para PDF escaneado
- Upload na interface (implementado e testado, mas desligado)
- Escala: um processo, um Chroma local

Regra prática

Use framework quando o objetivo é um protótipo rápido, o time precisa de muitos conectores e o risco de uma resposta errada é baixo.

Construa à mão quando é preciso provar o comportamento (recusa, citação, isolamento), auditar decisões, ensinar, ou quando dados sensíveis limitam o que pode sair da máquina.

Caminho do meio: framework para ingestão, conectores e observabilidade; código próprio e testado para o que precisa ser provado, como recusa, citação e isolamento entre bases.

A matriz do ensaio, medida com providers reais

Cenário	NVIDIA (llama-3.2-11b)	Gemini (3.5-flash)	OpenAI
Recuperação	2 citadas, 1 Recusa indevida	1 citada, 2× HTTP 429	3× HTTP 429
Citação	3 de 3 citadas (artigo jurídico)	1 citada, 2× HTTP 429	3× HTTP 429
Recusa	Recusa sem chamar geração	Recusa sem chamar geração	Recusa sem chamar geração
Filtro por ano	3 de 3 Recusa (esperado)	3× HTTP 429	3× HTTP 429
Filtro por tema	3 de 3 fallback, sem [n]	não medido (cota)	não medido

Três repetições por cenário; a Recusa roda uma vez porque não chama geração. Fonte: docs/evidencias/matriz-pt-br/.

Cadeia NVIDIA → Gemini → OpenAI: a NVIDIA respondeu os 5 cenários sem precisar de fallback. Nenhuma resposta citou fonte fora do filtro. O fallback de provider só foi observado de verdade numa medição anterior, no corpus antigo.

5 de 5

citações corretas na pergunta de citação (matriz, cadeia e UI)

3 de 6

Recusas indevidas na pergunta de recuperação, com retrieval 100% certo

3,3 s

mediana do 1º token na NVIDIA (0,8 a 20,9 s)

7 de 9

chamadas ao Gemini com HTTP 429; 1º token de 89 a 114 s quando respondeu

O que a evidência prova, e o que ela não prova

Prova

- Suíte de 119 testes passando
- Retry só antes do 1º token, troca por retry-after, Resposta Parcial sem troca
- Fonte Citada só com marcador; Recusa sem evidência
- Chave fora do log e da mensagem de erro final
- Retrieval e filtros no corpus pt-br; fluxo real pela UI com a NVIDIA

Não prova

- Geração real pela OpenAI (conta sem saldo, issue #70)
- Gemini funcionando como fallback (cota esgotada)
- Falha depois do 1º token com um provider real
- Fallback de provider observado pela interface
- Upload de PDFs pela interface (adiado)

O catálogo também tem limites

82

modelos listados em /models

410

llama-3.3-70b e 3.1-70b aposentados

404

nemotron-70b, mistral-large-2 e gemma-3 listados, mas não implantados

> 100 s

llama-3.2-90b-vision sem terminar

Segredos, logs e um rollback em três degraus

Rollback, do mais barato ao mais caro

1

Voltar ao corpus anterior

Restaurar o manifesto: a coleção de 661 chunks continua no Chroma. Minutos.

2

Trocar a ordem de providers

GENERATION_PROVIDERS_ORDER=gemini,nvidia,openai. Sem mudar código.

3

Voltar à geração local

git checkout legacy-pre-openai e reindexar: de 19 a 24 minutos. Nunca acontece automaticamente.

Nenhuma coleção foi apagada: a280e65e16ee, artigos_rag_openai e artigos_rag_openai_1789869651617850 seguem preservadas. A tag aponta para d68205d.

Segredos e dados

Chaves só em .streamlit/secrets.toml ou no ambiente; ambos no .gitignore

Varredura do histórico inteiro com git log --all -S, sem valor real encontrado

Evidências gravam só metadados: status, arquivo, página, provider e tempos

Logs nunca levam chave, prompt, texto de chunk ou de resposta

Erros externos são sanitizados: o corpo da resposta do provider não aparece na tela

Cinco perguntas antes da Parte 2

1

Recusar cedo ou deixar o LLM decidir?

No seu domínio, o que custa mais: uma recusa indevida ou uma resposta inventada?

2

Onde fica a linha entre evidência e citação?

Chunk recuperado deveria aparecer como fonte para o usuário final?

3

Que dados ainda saem da máquina?

Embeddings locais protegem o corpus, mas a pergunta e 3 trechos vão ao provider.

4

Quais metadados o seu corpus tem?

Filtrar antes da similaridade é a lição do RAG jurídico.

5

Framework ou Python puro?

Quando vale trocar controle e transparência por velocidade de montagem?

Na Parte 2

- Linha do tempo: quatro pivôs guiados por medição
- ADRs em cadeia, testes primeiro e glossário
- Engenharia com agentes de IA e gates de CTO
- Caso real de gate e duas rodadas de auditoria
- Debate: o que levar para os seus projetos